

VOZ DEL CIUDADANO

Plataforma de Iniciativas Legislativas Ciudadanas

Documentacion del Sistema - PC3 | Diseno de Software

Autores: Medrano & Giordano

Junio 2026

1. INTRODUCCION

El Legislativo de la Republica requiere el desarrollo de la plataforma digital 'Voz del Ciudadano' para automatizar el procesamiento de Iniciativas Legislativas Ciudadanas. El software permite a colectivos civiles crear propuestas normativas y recolectar firmas digitales de apoyo.

Al alcanzar el limite constitucional de 25 000 firmas digitales validas en un plazo maximo de 90 dias, el sistema congela criptograficamente el archivo (SHA-512) y lo envia automaticamente a la Oficina del Congreso para su distribucion a las comisiones parlamentarias. Los ciudadanos tambien pueden enviar comentarios, recursos de apoyo y modificaciones a las propuestas activas.

2. REQUISITOS FUNCIONALES

ID	Nombre	Descripcion
RF-01	Registro de Colectivo Civil	Registrarse con nombre, DNI del representante y correo electronico.
RF-02	Autenticacion de Usuarios	Autenticacion mediante JWT. Las operaciones requieren sesion iniciada.
RF-03	Creacion de Propuesta Legislativa	Un colectivo autenticado crea una propuesta. El sistema asigna plazo d
RF-04	Firma Digital de Propuesta	Un ciudadano autenticado firma una propuesta activa. Una firma por DNI
RF-05	Control de Plazo de 90 Dias	Verifica la fecha limite en cada firma. Propuestas vencidas pasan a EX
RF-06	Congelamiento Criptografico	Al alcanzar 25 000 firmas genera hash SHA-512 y marca la propuesta FRO
RF-07	Envio al Congreso	Tras el congelamiento envia el documento al Congreso y genera ticket d
RF-08	Gestion de Comentarios	Los usuarios autenticados agregan comentarios de texto a propuestas ac
RF-09	Recursos de Apoyo	Los usuarios adjuntan recursos (URL, titulo, tipo) que soportan la pro
RF-10	Consulta de Estado	Cualquier visitante consulta: firmas, progreso %, dias restantes y has

3. PATRONES ESTRUCTURALES IMPLEMENTADOS

3.1 Patron Facade - VozCiudadanaFacade

Proporciona una interfaz unificada y simplificada a los subsistemas de gestion de propuestas, recoleccion de firmas, congelamiento criptografico y envio al Congreso.

Clase principal: `app/patterns/facade.py` -> `VozCiudadanaFacade`

Metodos: `create_proposal()`, `sign_proposal()`, `_freeze_and_submit()`, `get_proposal_status()`

Beneficio: Los routers de FastAPI solo interactuan con la Facade, ocultando la complejidad interna del `CryptoService`, `CongressService` y las validaciones.

3.2 Patron Composite - LegislativeDocument

Construye el documento legislativo como un arbol de componentes: titulo, resumen, texto completo y firmas. Permite calcular propiedades del conjunto (`word_count`) y serializar el documento para el congelamiento criptografico.

Clases: `ProposalComponent` (abstracta), `ProposalLeaf`, `ProposalComposite`, `LegislativeDocument`

Ubicacion: `app/patterns/composite.py`

3.3 Patron Decorator - Cadena de Validadores de Firma

Añade capas de validacion en tiempo de ejecucion sin modificar la clase base.

Cada decorador valida un aspecto especifico de la operacion de firma:

- DeadlineValidatorDecorator : verifica que el plazo no haya vencido
- SignatureLimitDecorator : verifica que no se hayan alcanzado 25 000 firmas
- DuplicateSignatureDecorator : verifica que el ciudadano no haya firmado antes
- StatusValidatorDecorator : verifica que la propuesta este en estado COLLECTING

La funcion build_signature_validator() compone la cadena completa.

Ubicacion: app/patterns/decorator.py

4. CASOS DE USO

CU-01: Crear Iniciativa Legislativa

Actor principal:

Representante de Colectivo Civil (usuario autenticado)

Precondicion:

El usuario debe tener sesion iniciada (token JWT valido).

Flujo principal:

1. El colectivo accede a 'Nueva Propuesta'.
2. Ingresa titulo (min. 10 caracteres), resumen y texto completo.
3. El sistema valida los datos mediante el schema ProposalCreate.
4. La Facade crea la propuesta con status=COLLECTING y deadline=ahora+90 dias.
5. El sistema retorna la propuesta creada con su ID.

Postcondicion:

La propuesta queda publicada y abierta para recibir firmas.

Flujo alternativo:

- 3a. Titulo < 10 caracteres -> HTTP 422 con mensaje de error.
- 3b. Token JWT expirado -> HTTP 401 (no autorizado).

Patrones involucrados:

Facade (create_proposal), Composite (LegislativeDocument)

CU-02: Firmar una Propuesta Legislativa

Actor principal:

Ciudadano (usuario autenticado)

Precondicion:

La propuesta existe, tiene status=COLLECTING, no ha expirado y el ciudadano no la ha firmado previamente.

Flujo principal:

1. El ciudadano accede al detalle de la propuesta.
2. Hace clic en 'Firmar esta propuesta'.
3. La Facade ejecuta la cadena de Decorators de validacion.
4. Si todas las validaciones pasan, se registra la Signature en la BD.
5. Se incrementa signature_count. Si llega a 25 000 -> flujo de congelamiento.
6. El sistema retorna el nuevo conteo de firmas.

Postcondicion:

La firma queda registrada. Si se alcanzan 25 000, la propuesta es congelada y enviada al Congreso.

Flujo alternativo:

- 3a. Firma duplicada -> HTTP 400 'Ya firmaste esta propuesta'.
- 3b. Plazo vencido -> HTTP 400 'La propuesta ha expirado'.
- 3c. Propuesta congelada -> HTTP 400 'No se puede firmar'.

Patrones involucrados:

Facade (sign_proposal), Decorator (build_signature_validator), Composite (hash)

CU-03: Consultar Estado de Propuesta

Actor principal:

Cualquier visitante (sin autenticacion requerida)

Precondicion:

La propuesta debe existir en el sistema.

Flujo principal:

1. El visitante accede a la URL de detalle de la propuesta.
2. El sistema llama a `VozCiudadanaFacade.get_proposal_status()`.
3. Si `status=COLLECTING` y `deadline` ya paso, el sistema actualiza a `EXPIRED`.
4. Retorna: `id`, `title`, `status`, `signature_count`, `signatures_needed`, `progress_percent`, `days_remaining`, `crypto_hash`, `congress_ticket`.

Postcondicion:

El estado es devuelto al cliente. Si correspondia, la propuesta queda marcada como `EXPIRED`.

Flujo alternativo:

- 2a. Propuesta no encontrada -> HTTP 404.

Patrones involucrados:

`Facade (get_proposal_status)`

5. CASOS DE PRUEBA

CP-01: Creacion exitosa de propuesta legislativa

Objetivo:

Verificar que la Facade crea una propuesta con status=COLLECTING y plazo de 90 dias.

Precondicion:

Sistema iniciado. Usuario autenticado disponible.

Entrada:

title='Ley de Transparencia Digital', summary='...', full_text='...', author_id=1

Pasos:

1. Invocar VozCiudadanaFacade.create_proposal() con los datos anteriores.
2. Verificar el objeto retornado.
3. Verificar que LegislativeDocument construye el arbol Composite correctamente.

Resultado esperado:

proposal.status == 'collecting'

proposal.deadline aprox. utcnow + 90 dias

LegislativeDocument.build() retorna 4 nodos hijo | total_words > 0

Resultado:

PASS - TestCP01_CreacionPropuesta: todos los assertions pasan.

Archivo:

backend/tests/test_cases.py -> class TestCP01_CreacionPropuesta

CP-02: Firma numero 25 000 activa congelamiento automatico

Objetivo:

Verificar que la firma 25 000 dispara el congelamiento criptografico y el envio al Congreso.

Precondicion:

Propuesta con signature_count=24 999, status=COLLECTING, plazo vigente.

Entrada:

proposal_id=1, citizen_id=42, ip_address='127.0.0.1'

Pasos:

1. Crear mock de Proposal con signature_count=24 999.
2. Invocar VozCiudadanaFacade.sign_proposal().
3. Verificar que CryptoService genera el hash SHA-512.
4. Verificar que CongressService.submit() retorna un ticket.
5. Verificar el hash: 128 caracteres hexadecimales.

Resultado esperado:

result['success'] == True | '25,000' in result['message']

len(result['crypto_hash']) == 128 | result['congress_ticket'].startswith('CONGRESO-')

Resultado:

PASS - TestCP02_CongelamientoAutomatico: todos los assertions pasan.

Archivo:

backend/tests/test_cases.py -> class TestCP02_CongelamientoAutomatico

CP-03: Firma rechazada en propuesta expirada (90 dias vencidos)

Objetivo:

Verificar que el sistema rechaza firmas cuando el plazo de 90 dias ha vencido.

Precondicion:

Propuesta con deadline=ahora-1 dia, status=COLLECTING.

Entrada:

deadline=utcnow()-timedelta(days=1), signature_count=5000, citizen_id=10

Pasos:

1. Construir la cadena de Decorators con build_signature_validator().
2. Ejecutar validator.execute(data) con deadline expirado.
3. Invocar get_proposal_status() con propuesta expirada hace 91 días.
4. Verificar que el estado cambia a EXPIRED.

Resultado esperado:

result['success'] == False | 'expirado' in result['error']

proposal.status == ProposalStatus.EXPIRED | days_remaining == 0

Resultado:

PASS - TestCP03_PropuestaExpirada: todos los assertions pasan.

Archivo:

backend/tests/test_cases.py -> class TestCP03_PropuestaExpirada

6. ARQUITECTURA DEL SISTEMA

Arquitectura de tres capas con separacion clara de responsabilidades:

FRONTEND (React JS, puerto 3000)

pages/ : Home, ProposalList, ProposalDetail, CreateProposal, Login, Register

components/ : Navbar, ProposalCard, SignatureCounter, CommentSection

services/ : api.js - llamadas HTTP con Axios + JWT

BACKEND (FastAPI + Python 3.13, puerto 8000)

routers/ : users.py, proposals.py (endpoints REST)

patterns/ : facade.py, composite.py, decorator.py

services/ : crypto_service.py, congress_service.py

models/ : SQLAlchemy ORM (User, Proposal, Signature, Comment, Resource)

schemas/ : Pydantic v2 (validacion de entrada/salida)

database.py : SQLite en desarrollo, configurable via DATABASE_URL

BASE DE DATOS

SQLite para desarrollo, compatible con PostgreSQL en produccion.

7. MODELO DE BRANCHING - GITFLOW

Ramas del repositorio:

main : codigo en produccion
 develop : integracion de funcionalidades
 feature/backend-setup : configuracion inicial FastAPI + modelos
 feature/structural-patterns : Facade, Composite, Decorator
 feature/frontend-react : implementacion React JS
 feature/tests : casos de prueba pytest
 release/1.0 : preparacion del release final

Repositorio: <https://github.com/shordanooo/medranogiordano-pc3>

8. INSTRUCCIONES DE EJECUCION

BACKEND:

```
cd backend
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload --port 8000
Swagger UI: http://localhost:8000/docs
```

FRONTEND:

```
cd frontend
npm install
npm start
Aplicacion: http://localhost:3000
```


TESTS:

cd backend

pytest tests/ -v